2026-8-15 Linux线程--同步

- 1. 从互斥走向线程同步
  - 1.1 互斥保证安全，同步保证顺序
    - 1.1.1 同步与互斥的区别
    - 1.1.2 竞态条件
- 2. 条件变量
  - 2.1 为什么需要条件变量
  - 2.2 pthread_cond_wait 的等待语义
    - 2.2.1 为什么等待后还要重新判断条件
    - 2.2.2 等待、通知与临界区的关系
  - 2.3 条件变量接口
- 3. 生产者消费者模型
  - 3.1 一个典型的线程同步场景
  - 3.2 生产者消费者模型的“321”原则
  - 3.3 为什么使用生产者消费者模型
- 4. 用 BlockingQueue 实现生产消费
  - 4.1 阻塞队列的成员设计
  - 4.2 第一版：用 if 检查等待条件
    - 4.2.1 第一版存在的问题
  - 4.3 优化版：while 重检与等待计数
    - 4.3.1 while 如何消除错误唤醒
  - 4.4 单生产、单消费示例
- 5. 运行现象与同步关系
  - 5.1 缓冲区先被填满，再交替推进
  - 5.2 生产与消费可以一对一推进
- 6. 条件变量使用规范
  - 6.1 等待与通知必须围绕同一份共享状态
  - 6.2 条件变量与互斥量的分工
  - 6.3 线程同步的核心结论

1. 从互斥走向线程同步

互斥解决了“多个线程不能同时修改共享资源”的问题，但没有解决“线程应该按什么顺序等待和继续”的问题。线程同步是在保证临界资源安全的前提下，让线程按照确定的先后关系访问资源，从而避免无意义地反复检查、长期等待或某个线程一直抢不到执行机会。

1.1 互斥保证安全，同步保证顺序

可以用自习室来理解：

- 自习室是多个线程共享的临界资源。
- 进入自习室的过程是临界区。
- 钥匙相当于互斥锁，同一时刻只有一把钥匙。
- 人相当于竞争资源的线程。

互斥只能保证同一时刻只有一个人进入，却没有规定归还钥匙后谁应该先进入。如果刚出来的人立刻又把钥匙抢走，外面排队的人可能一直进不去，这就是只有互斥、没有同步时可能出现的不合理竞争。

因此还需要一条排队规则：

1. 归还钥匙后，如果还想进入，必须重新排队。
2. 已经等待的外部线程也必须按照竞争规则排队。

一句话概括：**在保证数据安全的前提下，让线程按照某种特定顺序访问临界资源，并避免饥饿。**

1.1.1 同步与互斥的区别

机制	主要解决的问题
互斥	同一时刻不能让多个线程同时进入临界区
同步	条件不满足时应该等待，条件满足后按合理顺序继续

互斥是同步的前提。没有互斥，条件判断和共享数据的修改本身就可能相互破坏；只有互斥，又容易变成无休止的竞争或忙等。

1.1.2 竞态条件

程序的正确性如果依赖线程恰好按照某个时序执行，就会存在竞态条件。线程调度的顺序并不固定，同一份代码今天运行正确，换一种调度顺序后可能就得到不同结果。

因此程序不能依赖“线程通常会这样执行”，而应通过互斥和同步明确表达线程之间必须满足的先后关系。

2. 条件变量

2.1 为什么需要条件变量

互斥锁适合保护共享数据，却不适合表达“数据现在还不能使用”。例如消费者访问队列时，如果队列为空，它即使拥有锁也做不了任何事：

- 一直持有锁忙等，生产者就无法进入临界区放入数据，消费者也永远等不到数据。
- 不断加锁、检查、解锁、再检查，会持续消耗 CPU，而且线程仍可能长期抢不到锁。
- 如果只根据一次判断就继续执行，又可能在前一次唤醒后条件已经不再成立。

条件变量用于表达“线程正在等待某个条件成立”。消费者等待“队列非空”，生产者等待“队列未满”；条件满足后，由改变共享状态的线程发出通知。

> 条件变量本身不保存条件，也不是一个布尔变量。真正的条件是共享状态，例如 `_bq.empty()`、`_bq.size() == _cap`。条件变量负责等待与通知，互斥锁负责保护这些状态。

2.2 `pthread_cond_wait` 的等待语义

条件变量最关键的接口是：

```
int pthread_cond_wait(
    pthread_cond_t *restrict cond,
    pthread_mutex_t *restrict mutex);
```

调用后依次发生以下过程：

1. 当前线程必须已经持有 `mutex`。
2. `pthread_cond_wait` 原子地释放 `mutex` 并阻塞，让其他线程可以修改共享状态并发出通知。
3. 被唤醒后，线程先重新竞争并取得 `mutex`，然后才从 `pthread_cond_wait` 返回。

这里必须传入互斥量，是因为“检查条件”和“进入等待”之间不能留下空档。如果先手动解锁再等待，其他线程可能恰好在两步之间修改状态并发出通知，当前线程随后却错过了通知，一直

睡下去。

2.2.1 为什么等待后还要重新判断条件

条件变量允许伪唤醒，且一个通知可能对应多个等待者。被唤醒只表示“可以重新检查条件”，并不保证条件一定为真。

例如消费者被唤醒时，另一个消费者可能已经先取得锁并取走了唯一的元素。第一个消费者重新取得锁后，队列仍然是空的，所以不能直接执行 `pop`。

正确模式是使用 `while`：

```
pthread_mutex_lock(&mutex);
while (condition_is_false) {
    pthread_cond_wait(&cond, &mutex);
}
// 离开循环时，条件一定成立
pthread_mutex_unlock(&mutex);
```

使用 `if` 只检查一次条件，无法处理伪唤醒或唤醒后的再次竞争。

2.2.2 等待、通知与临界区的关系

等待条件的代码必须位于互斥锁保护的区域中，因为检查共享队列本身就是访问临界资源。只有把“检查条件”和“等待”放在同一把锁的保护下，才能避免丢失通知。

通知本身可以在临界区内也可以在临界区外发出，但共享状态必须在解锁前修改。实际代码通常先把数据放入或移出队列，再通知对端，最后解锁。

2.3 条件变量接口

初始化条件变量有两种方式。

静态初始化：

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

动态初始化：

```
int pthread_cond_init(
    pthread_cond_t *restrict cond,
    const pthread_condattr_t *restrict attr);
```

其中 `attr` 传 `nullptr` 表示使用默认属性。

销毁条件变量：

```
int pthread_cond_destroy(pthread_cond_t *cond);
```

通知接口：

```
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

- `pthread_cond_signal`：至少唤醒一个等待该条件变量的线程。
- `pthread_cond_broadcast`：唤醒所有等待该条件变量的线程。
- 被唤醒的线程仍然需要重新获取互斥锁，并且重新检查条件。

对“队列非空”和“队列未满”这类方向相反的条件，应该分别使用两个条件变量，避免生产者和消费者被错误地唤醒。

3. 生产者消费者模型

3.1 一个典型的线程同步场景

生产者线程生成数据并放入共享缓冲区，消费者线程从缓冲区取出数据并处理。双方不直接调用彼此，而是通过共享缓冲区通信：

```
生产者线程   --->   阻塞队列   --->   消费者线程
   生产                缓存/缓冲              消费
```

阻塞队列相当于仓库或超市：

- 队列为空时，消费者必须等待生产者放入数据。
- 队列已满时，生产者必须等待消费者取走数据。
- 队列未满且非空时，生产者和消费者可以继续并发工作。

3.2 生产者消费者模型的“321”原则

三个关系：

线程关系	需要的约束
生产者与生产者	互斥访问队列尾部，防止同时写入破坏队列
消费者与消费者	互斥访问队列头部，防止同时取走同一元素
生产者与消费者	既需要互斥访问队列，又需要同步等待可用空间或数据

两个角色：生产者、消费者。

一个交易场所：共享缓冲区，通常由队列等数据结构实现。

3.3 为什么使用生产者消费者模型

生产者消费者模型通过中间缓冲区降低生产者和消费者之间的强耦合：

- **解耦**：生产者不需要知道消费者何时处理，消费者也不需要直接向生产者索取。
- **支持并发**：生产者生成数据与消费者处理数据可以同时进行。
- **平衡忙闲不均**：某一方暂时更快时，队列可以吸收速度差，避免另一方被迫同步等待。

生产者生产完数据后可以继续生产，消费者从队列中取数据后独立处理。缓冲区把双方的直接依赖转换为对共享队列的协调。

4. 用 `BlockingQueue` 实现生产消费

4.1 阻塞队列的成员设计

一个线程安全的有界阻塞队列至少需要以下成员：

```
std::queue<T> queue;                    // 共享缓冲区
std::size_t capacity;                   // 容量
```

```
pthread_mutex_t mutex;              // 保护队列及等待计数
pthread_cond_t not_empty;           // 消费者等待队列非空
pthread_cond_t not_full;            // 生产者等待队列未满
int waiting_consumers = 0;          // 正在等待数据的消费者数量
int waiting_producers = 0;          // 正在等待空间的生产者数量
```

队列为空或已满并不是两把不同的锁，而是同一个共享状态在两种条件不满足时的表现。`mutex` 负责数据安全，两个条件变量负责两个方向的等待与通知。

4.2 第一版：用 `if` 检查等待条件

第一版通常会写出类似下面的结构：

```
void Push(T& value)
{
    pthread_mutex_lock(&mutex);
    if (queue.size() == capacity) {
        pthread_cond_wait(&not_full, &mutex);
    }

    queue.push(value);
    pthread_cond_signal(&not_empty);
    pthread_mutex_unlock(&mutex);
}

void Pop(T* out)
{
    pthread_mutex_lock(&mutex);
    if (queue.empty()) {
        pthread_cond_wait(&not_empty, &mutex);
    }

    *out = queue.front();
    queue.pop();
    pthread_cond_signal(&not_full);
    pthread_mutex_unlock(&mutex);
}
```

这段代码能够演示最基本的同步关系：队列满时生产者等待，队列空时消费者等待。但 `if` 只判断一次，不适合作为最终版本。

4.2.1 第一版存在的问题

线程从 `pthread_cond_wait` 返回，并不代表它等待的条件一定成立。可能发生以下情况：

- 线程被伪随机唤醒，实际上没有数据可消费。
- 多个消费者同时被唤醒，第一个消费者取走元素后，队列再次为空。
- 多个生产者同时被唤醒，第一个生产者填满队列后，其他生产者已没有可用空间。

如果唤醒后不重新检查条件，消费者可能读取空队列，生产者可能向已满队列继续写入。这种错误依赖调度概率，往往不是每次运行都会出现。

4.3 优化版：`while` 重检与等待计数

优化版需要做三件事：

1. 用 `while` 重新检查条件。
2. 记录等待线程数量，只有确实有人等待时才发通知。
3. 在状态改变后检查对端等待者，减少无效唤醒。

```
#ifndef BLOCK_QUEUE_HPP
#define BLOCK_QUEUE_HPP

#include <cstddef>
#include <queue>
#include <pthread.h>

template <typename T>
class BlockQueue {
public:
    explicit BlockQueue(std::size_t capacity = 5)
        : capacity_(capacity)
    {
        pthread_mutex_init(&mutex_, nullptr);
        pthread_cond_init(&not_empty_, nullptr);
        pthread_cond_init(&not_full_, nullptr);
    }

    BlockQueue(const BlockQueue&) = delete;
    BlockQueue& operator=(const BlockQueue&) = delete;

    void Push(const T& value)
    {
        pthread_mutex_lock(&mutex_);
```

```
        while (queue_.size() == capacity_) {
            ++waiting_producers_;
            pthread_cond_wait(&not_full_, &mutex_);
            --waiting_producers_;
        }

        queue_.push(value);

        if (waiting_consumers_ > 0) {
            pthread_cond_signal(&not_empty_);
        }

        pthread_mutex_unlock(&mutex_);
    }

    T Pop()
    {
        pthread_mutex_lock(&mutex_);

        while (queue_.empty()) {
            ++waiting_consumers_;
            pthread_cond_wait(&not_empty_, &mutex_);
            --waiting_consumers_;
        }

        T value = queue_.front();
        queue_.pop();

        if (waiting_producers_ > 0) {
            pthread_cond_signal(&not_full_);
        }

        pthread_mutex_unlock(&mutex_);
        return value;
    }

    ~BlockQueue()
    {
        pthread_mutex_destroy(&mutex_);
        pthread_cond_destroy(&not_empty_);
        pthread_cond_destroy(&not_full_);
    }

private:
    std::queue<T> queue_;
    std::size_t capacity_;
```

```
    pthread_mutex_t mutex_;
    pthread_cond_t not_empty_;
    pthread_cond_t not_full_;
    int waiting_consumers_ = 0;
    int waiting_producers_ = 0;
};

#endif
```

4.3.1 `while` 如何消除错误唤醒

消费者被唤醒后重新取得锁，并再次判断 `queue_.empty()`：

- 如果其他消费者已经取走数据，条件仍然为假，当前消费者继续等待。
- 如果队列中确实有数据，才退出循环并执行 `pop`。

生产者同理。被唤醒后重新判断 `queue_.size() == capacity_`，只有队列确实存在空闲位置时才执行 `push`。

等待计数在持有互斥锁时修改，因此它也属于临界资源。发送通知前检查计数大于零，可以避免对没有等待者的条件变量做无效通知。

4.4 单生产、单消费示例

下面的主程序让生产者不断写入递增整数，消费者稍慢地取出并打印：

```
#include <iostream>
#include <unistd.h>
#include <pthread.h>
#include "BlockQueue.hpp"

void* ConsumerRoutine(void* args)
{
    auto* queue = static_cast<BlockQueue<int>*>(args);

    while (true) {
        sleep(5);
        int data = queue->Pop();
        std::cout << "消费 " << data << std::endl;
    }
    return nullptr;
```

```
}

void* ProducerRoutine(void* args)
{
    auto* queue = static_cast<BlockQueue<int>*>(args);
    int data = 10;

    while (true) {
        queue->Push(data);
        std::cout << "生产 " << data++ << std::endl;
    }
    return nullptr;
}

int main()
{
    BlockQueue<int> queue(5);

    pthread_t consumer;
    pthread_t producer;
    pthread_create(&consumer, nullptr, ConsumerRoutine, &queue);
    pthread_create(&producer, nullptr, ProducerRoutine, &queue);

    pthread_join(consumer, nullptr);
    pthread_join(producer, nullptr);
    return 0;
}
```

生产者不需要等待消费者立即处理，只需要把数据写入队列；消费者也不需要直接联系生产者，只需要在队列非空时取出数据。双方通过同一个阻塞队列完成同步。

5. 运行现象与同步关系

5.1 缓冲区先被填满，再交替推进

消费者执行 `sleep(5)` 时，生产者可以连续生产，直到队列达到容量上限后阻塞。消费者取走一个元素并通知生产者后，生产者才能继续写入，因此会形成“先积累一段数据，之后生产和消费交替推进”的输出：

```
ZL@VM-0-14-ubuntu:~/code/lesson32/1.BlockQueue$ ./cp_bq
生产10
生产11
生产12
生产13
生产14
消费10
生产15
消费11
生产16
消费12
生产17
消费13
生产18
消费14
生产19
```

图中的先后顺序不是简单随机结果，而是由两个条件决定的：

- 队列满时，生产者停在 `not_full` 上等待。
- 队列空时，消费者停在 `not_empty` 上等待。

5.2 生产与消费可以一对一推进

如果生产者和消费者的速度较为接近，或者初始时序不同，就可能观察到接近“生产一个、消费一个”的交替顺序：

```
ZL@VM-0-14-ubuntu:~/code/lesson32/1.BlockQueue$ ./cp_bq
生产10
消费10
生产11
消费11
生产12
消费12
生产13
消费13
生产14
消费14
生产15
消费15
生产16
消费16
生产17
消费17
```

这里的“交替”不是要求生产者和消费者绑定执行，而是条件变量把双方组织成了合理的等待与唤醒顺序。只要队列既有空间又有数据，双方具体谁先执行仍由调度决定。

6. 条件变量使用规范

6.1 等待与通知必须围绕同一份共享状态

条件变量的正确使用可以归纳为：

1. 加锁。
2. 用 `while` 检查共享条件。
3. 条件不满足时调用 `pthread_cond_wait`。
4. 条件满足后修改共享状态。
5. 必要时通知等待对端。
6. 解锁。

条件变量本身不知道“空”或“满”的含义，它只负责挂起和唤醒。队列状态必须由互斥锁保护，条件变量只是让线程在状态不满足时放弃 CPU 等待。

6.2 条件变量与互斥量的分工

- **互斥量**回答“现在能不能安全地检查或修改共享数据”。
- **条件变量**回答“条件暂时不满足时，线程应该在哪里等待，以及什么时候再检查”。

两者缺一不可：

- 只有互斥量：线程可能不断争抢锁检查条件，形成忙等，或者长期占用锁导致对方无法推进。
- 只有条件变量：共享状态失去保护，检查、修改和通知之间可能发生竞态。
- 互斥量加条件变量：既保证数据安全，又让线程在条件不满足时以阻塞方式等待。

6.3 线程同步的核心结论

- 互斥强调“同一时刻只能有一个线程进入临界区”，同步强调“线程应按合理顺序等待和继续”。
- 条件变量不保存条件，真正的判断依据仍是受互斥锁保护的共享状态。
- `pthread_cond_wait` 会原子地释放互斥量，并在返回前重新取得互斥量。
- 等待条件必须使用 `while` 循环重检，不能依赖一次 `if` 或一次通知。
- 生产者消费者模型用一块共享缓冲区连接两类线程，生产中需要等待空间，消费中需要等待数据。

- 阻塞队列同时使用“一把互斥锁 + 两个条件变量”，既解决生产者和消费者之间的同步，也解决同类线程之间的互斥。